Flutter Course

OOP

Ruaa Alshehabi

Intro to OOP

- Object-Oriented Programming is a programming paradigm based on the concept of "objects", which can contain data, in the form of fields or attributes, and code, in the form of procedures or methods.

- By utilizing these concepts, Object-Oriented Programming helps in creating modular, maintainable, and scalable code. It promotes a clear separation of concerns and allows for easier collaboration in larger software projects.

- OOP have four concepts :

Class and Object

1-Encapsulation

2- Inheritance

3-Polymorphism

4-Abstraction

Class and Object

1. A class is like a blueprint for creating objects. It defines the properties (attributes) and behaviors (methods) that all objects of that type will have.
2. instance of a class. It represents a specific instance of the class, with its own unique set of data and behavior.

```
class Human {
  String? name;
  int? age;
  double? height;
  double? wight;
}

void main() {
  Human human = Human();

  human.name = 'Mohamed';
  human.age = 32;
  human.height = 169.1;
  human.wight = 70.5;
}
```

Methods

- methods within a class are defined similarly to regular functions, but they are indented within the class definition

```
class Human {
  String? name;
  int? age;
  double? height;
  double? wight;

  void run()
  {
    print("Running....");
  }
}

void main() {
  Human human = Human();

  human.name = 'Mohamed';
  human.age = 32;
  human.height = 169.1;
  human.wight = 70.5;
  human.run();
}
```

Constructors

- In Dart, the constructor is a special method within a class that is automatically called when an object of that class is created. It is used to initialize the object's state, which often involves assigning values to the object's fields (properties).

```
class Human {
  String? name;
  int? age;
  double? height;
  double? wight;

  Human() {

  }

  void run() {
    print("Running....");
  }
}
```

Named constructors

- Named constructors are a type of constructor that allows you to define multiple constructors for a class, each with a unique name. Named constructors are particularly useful when you want to provide different ways to create instances of a class with different sets of parameters or initialization logic.

```
void main() {

Circle circle1 =Circle(radius: 5, x: 1, y: 2);
Circle circle2 =Circle.origin(radius: 5, );

print(circle1.draw());
print(circle2.draw());
}
class Circle {
  double radius;
  int?x ;
  int?y;

  String draw ()=>"Drawing the circle with x = $x and y = $y with radius: $radius";

  Circle ({required this.radius, required this.x, required this.y});

  Circle.origin({required this.radius}){
    x = 0;
    y=0;
  }
}
```

Attributes

- Attributes, also known as properties or fields, are the data components of a class that represent the characteristics or state of objects created from that class. In Object-Oriented Programming (OOP), attributes define the structure of objects.

```
class Car {
  // Attributes
  String make;
  String model;
  int year;
  bool isRunning;

  // Constructor
  Car(String make, String model, int year) {
    // Attribute initialization
    this.make = make;
    this.model = model;
    this.year = year;
    isRunning = false;
  }

  // Methods...
}
```

Constructors 2

- Constructors are used to simplify the value that assigned to attribute so with constructors you can path the value with one single line of code.

```
class Human {
  String? name;
  int? age;
  double? height;
  double? weight;

  Human(
      {required String this.name, required int this.age, required double this.height, required double this.weight});

  void run() {
    print("Running....");
  }
}

void main() {
  Human human = Human(name: "Mohamed", age: 32,height: 171.2,weight: 70.2);
  print(human.name);


  human.run();
}
```

Task1

- **Define a class named Task with the following attributes:**
- title (String)
- description (String)
- Date (String)
- **Create a constructor for the Task class** to initialize these properties.
- **Implement a method displayTask** in the Task class to print the task details.

1-Encapsulation

- Encapsulation involves hiding the internal state of an object, including its attributes, to prevent direct access and manipulation from outside the class.

```dart
class Human {
  String? name;
  int? age;
  int _numberOfArms =2;
  double? height;
  double? weight;

  Human(
      {required String this.name, required int this.age, required double this.height, required double this.weight});

  void run() {
    print("Running....");
  }
}
```

```dart
import 'human_class.dart';

void main() {
  Human human = Human(name: "Mohamed", age: 32,height: 171.2,weight: 70.2,);
  human._numberOfArms =10;
  print(human.name);


  human.run();
}
```

How to access private attributes

- By encapsulating attributes and providing controlled access through methods, you can ensure that the object's data remains consistent and valid.

```dart
class Human {
  String? name;
  int? age;
  int _numberOfArms =2;
  double? height;
  double? weight;

  Human(
      {required String this.name, required int this.age, required double this.height, required double this.weight});

  void run() {
    print("Running....");
  }

  void setNumberOfArms(int numberOfArms)
  {
    if(numberOfArms<=2&&numberOfArms>=0)
      {
        _numberOfArms = numberOfArms;
      }
  }
  int getNumberOfArms()
  {
    return _numberOfArms;
  }
}
```

```dart
import 'human_class.dart';

void main() {
  Human human = Human(name: "Mohamed", age: 32,height: 171.2,weight: 70.2,);

  human.setNumberOfArms(1);

  print(human.getNumberOfArms());

}
```

Setter

- Setter is used to assign the value to a private attribute without using methods and don't care about return value.

```dart
class Human {
  String? name;
  int? age;
  int _numberOfArms =2;
  double? height;
  double? weight;

  Human(
      {required String this.name, required int this.age, required double this.height, required double this.weight});

  void run() {
    print("Running....");
  }

  set setNumberOfArms(int numberOfArms)
  {
    if(numberOfArms<=2&&numberOfArms>=0)
      {
        _numberOfArms = numberOfArms;
      }
  }
  int getNumberOfArms()
  {
    return _numberOfArms;
  }
}
```

```dart
import 'human_class.dart';

void main() {
  Human human = Human(name: "Mohamed", age: 32,height: 171.2,weight: 70.2,);

  human.setNumberOfArms = 1;

  print(human.getNumberOfArms());

}
```

Arrow Function

- Is used instead of {} brackets if the function will return one statement.

```
  //Normal function
  int getNumberOfArms()
  {
    return _numberOfArms;
  }
//Arrow Function
  int numbersOfArms () => _numberOfArms;
}
```

Getter

- Getter is used to access and return the value to a private attribute without using methods and don't care about parameters.

```dart
set setNumberOfArms(int numberOfArms) {
  if (numberOfArms <= 2 && numberOfArms >= 0) {
    _numberOfArms = numberOfArms;
  }
}

int get getNumbersOfArms=> _numberOfArms;


}
```

```dart
import 'human_class.dart';

void main() {
  Human human = Human(name: "Mohamed", age: 32,height: 171.2,weight: 70.2,);

  human.setNumberOfArms = 1;

  print(human.getNumbersOfArms);

}
```

More About Constructors

- We can remove the {} brackets and replace this keyword before the attribute to assign the value to the attribute direct without using other values.

```
Human({required String name, required int age, required double height, required double weight}){
  this.name = name;
  this.age = age;
  this.height = height;
  this.weight = weight;
}
```

```
Human({required String this.name, required int this.age, required double this.height, required double this.weight});
```

2-Inheritnce

- Allows one class to inherit properties and behaviors (methods) from another class. inheritance enables code reuse and promotes a hierarchical organization of classes.

```
void main() {
  Dog jack = Dog();
  Cat kitty = Cat();
  Lion scar = Lion();

  print(jack.numOfLimbs);

}

class Animal {
  int numOfLimbs = 4;
  eat()=> print('eating');

}
class Dog extends Animal {

}
class Cat extends Animal {

}

class Lion extends Animal{

}
```

Super Constructor

- Is used to pass data from a subclass to its superclass (base class) when the subclass inherits from it. It allows the subclass constructor to invoke the constructor of its superclass, providing necessary data for initializing inherited members.

```
void main() {
  Dog jack = Dog(30,"black");
  Cat kitty = Cat(20,"white");
  Lion scar = Lion(60,"Brown");

  print(jack.numOfLimbs);
  print(kitty.skinColor);
  print(scar.weight);

}

class Animal {
  int numOfLimbs = 4;
  eat()=> print('eating');
  double ? weight;
  String ? skinColor;
Animal(this.weight, this.skinColor);
}

class Dog extends Animal {
  Dog(super.weight, super.skinColor);

}
class Cat extends Animal {
  Cat(super.weight, super.skinColor);

}

class Lion extends Animal{
  Lion(super.weight, super.skinColor);

}
```

Super Constructor : is used to pass the values from sub class to super class Like normal constructor but you can add an attribute from the sub class and use it without pass to Super class

```
class Animal {
  int numOfLimbs = 4;
  eat()=> print('eating');
  double ? weight;
  String ? skinColor;
Animal({required this.weight, required this.skinColor});
}

class Dog extends Animal {
  String bloodType;
  Dog({required this.bloodType ,required double weight, required String skinColor}) : super(skinColor: skinColor, weight: weight);

}
class Cat extends Animal {
  Cat({required super.weight, required super.skinColor});

}

class Lion extends Animal{
  Lion({required super.weight, required super.skinColor});

}
```

Override

- Is used to over-ride the implementation of a method in the super class and use it in the sub class with different implementation but must be with the same name and add word Override.

```
class Animal {
  int numOfLimbs = 4;
  eat()=> print('eating');
  double ? weight;
  String ? skinColor;
Animal({required this.weight, required this.skinColor});
}

class Lion extends Animal {
  Lion({required super.weight, required super.skinColor});

  @override
  String eat() {
    return "lion is eating"; // Return a value from the overridden eat() method
  }
}

class Dog extends Animal {...}
class Cat extends Animal {...}
```

3- Polymorphism

- allows objects of different classes to be treated as objects of a common superclass. It enables a single interface to represent multiple underlying data types or forms, leading to more flexible and reusable code. To apply polymorphism we must use concept inheritance.

```
void main() {
 Dog jack = Dog(weight: 30, skinColor: 'black',);
 Cat kitty = Cat(weight: 20, skinColor: 'white',);
 Lion scar = Lion(weight: 60, skinColor: 'brown',);
  List<Animal> animals = [jack, kitty,scar];

  print(animals[1].weight);
}

class Animal {
  int numOfLimbs = 4;
  String eat() =>"eating";
  double ? weight;
  String ? skinColor;


  Animal({required this.weight, required this.skinColor
}
class Dog extends Animal {...}

class Lion extends Animal {...}

class Cat extends Animal {...}
```

4- Abstraction

- An abstract class is a class that cannot be instantiated directly. It serves as a template or blueprint for other classes and may contain one or more abstract methods. Abstract classes are meant to be subclassed, and they can provide default implementations for some methods.

```
abstract class Animal {
  int numOfLimbs = 4;
 String eat();
  double ? weight;
  String ? skinColor;
Animal({required this.weight, required this.skinColor});
}

class Lion extends Animal {
  Lion({required super.weight, required super.skinColor});

  @override
  String eat() {
    return "lion is eating";
  }
}
```

Implements vs extend

- **extends** is used to establish an inheritance relationship between classes, allowing subclassing and inheriting behavior from a superclass.
- **implements** is used to enforce that a class adheres to a certain contract defined by an interface, enforce the class to provide concrete implementations for all methods declared in the interface.

```
abstract class Animal {
  int numOfLimbs = 4;
 String eat();
  double ? weight;
  String ? skinColor;
Animal({required this.weight, required this.skinColor});
}
class Dog implements Animal {
  String bloodType;
  Dog({required this.bloodType }) : super( );

  @override
  String eat() {
    return 'dog is eating';
  }

  @override
  int numOfLimbs = 6;

  @override
  String? skinColor;

  @override
  double? weight = 60 ;

}
```

Mixin

- Mixin is a way to add functionality to a class without using inheritance. Mixins allow you to reuse code across multiple classes by combining the members (methods and properties) of one class into another class.

```
void main() {
  Dog jack = Dog();
  Snake w = Snake();
  jack.numberOfLimbs;
  jack.walk();
  w.crawl();
}

class Animal {
  int numberOfLimbs = 4;
}

mixin MammalsMixin {
  walk() {}
}

mixin ReptilesMixin {
  crawl() {}
}

class Dog extends Animal with MammalsMixin {}

class Cat extends Animal with MammalsMixin {}

class Snake extends Animal with ReptilesMixin {}
```

Anonymous objects

- Anonymous objects refer to instances of classes that are created without assigning them to a variable. Instead, they are instantiated and used directly within the context where they are created. Anonymous objects are particularly useful when you need to perform a short-lived operation or when you don't need to reference the object multiple times throughout your code.

```
void main() {
  Dog jack = Dog(50);
  Cat(30);
  jack.weight;

  List<Animal> animals = [jack, Cat(30)];

  print(animals[1].weight);
}

class Animal {
  int weight;

  Animal(this.weight);
}

class Dog extends Animal {
  Dog(super.weight);
}

class Cat extends Animal {
  Cat(super.weight);
}
```

Task 2

- Create a BankAccount class that encapsulates the account balance. Provide methods to deposit, withdraw, and check the balance, but prevent direct access to modify the balance. Create Set and Get (return balance);

- Create a Person class and a Student class that inherits from Person. Add specific properties and methods to the Student class. (name , age , address) - student(grade , degree, gender)

- Create an abstract class Shape with an abstract method area(). Then create two subclasses Circle and Rectangle that implement the area method.

- Create a method printArea that takes a Shape object and prints its area. Demonstrate polymorphism by passing different shapes to this method.